

Lab work week4

Student Full Name: Ainat Yermurat

Student ID Number: 230103161

Institutional Email / Git URL: 230103161@sdu.edu.kz

<https://github.com/yermy-bc/Tasks-parallel-computing-314> Practicum Date & Time (Slot):
25.09.2026 14:00

Laptop Model & OS (Linux/macOS/Win):Lenovo Legion Ideapad gaming 3 & Windows 11

CPU Model & Generation:11th Gen Intel(R) Core(TM) i5-11300H @ 3.10GHz

Physical Cores (P/E breakdown if hybrid):4

Logical SMT Threads (omp_get_max_threads): 8

Target Workload Size N (Derived via Formula): 13,161,000 L1 / L2 Cache Line Size
(Typically 64 bytes): 64 bytes

Task 1.1 int txt

Task 1.2

=====

TASK 1.2: THREAD OVERSUBSCRIPTION SWEEP

=====

CPU logical processors: 8

P	Average Time (s)	Minimum Time (s)
1	0.00026363	0.00011970
2	0.00024141	0.00019830
4	0.00045758	0.00040640
8	0.00087809	0.00082150
16	0.00176812	0.00167680
32	0.00346278	0.00335590
64	0.00812493	0.00682250

Results saved to: task1_2_results.csv

Process finished with exit code 0

Task 1.3

=====

TASK 1.3: CPU CORE SATURATION ANALYSIS

=====

Logical CPU cores: 8

Workload per thread: 10,000,000 square roots

P	Execution Time (s)
Running P = 1 ...	
1	1.0225
Running P = 2 ...	
2	1.8808
Running P = 4 ...	

4	3.6073
Running P = 8 ...	
8	7.1875
Running P = 16 ...	
16	14.5310
Running P = 32 ...	
32	29.0644
Running P = 64 ...	
64	58.5463

Results saved to: task1_3_results.csv

Process finished with exit code 0

Question 1.1 (Scheduling)

Thread IDs are printed in a non-sequential order because the operating system scheduler determines when each thread gets CPU execution time. Thread creation order does not guarantee execution order. After the threads are created, the OS scheduler assigns CPU time to runnable threads according to scheduling policies, CPU availability, priorities, and system load.

The CPU architecture also affects the observed order because multiple hardware execution threads may execute simultaneously on different CPU cores. As a result, one thread created later may start executing before another thread created earlier.

Therefore, the output order can change between consecutive runs because thread scheduling is nondeterministic from the application's point of view. The main components responsible for this behavior are the **operating system scheduler** and the **CPU's hardware execution resources**.

Question 1.2 (Oversubscription)

Oversubscription occurs when the number of software threads (P) exceeds the number of available physical CPU cores. In this situation, the operating system cannot execute all threads simultaneously on separate physical cores. Instead, it must repeatedly switch between runnable threads.

This creates a **context-switching overhead**. During a context switch, the operating system saves the execution state of the currently running thread and restores the state of another thread. The saved state includes information such as registers, the program counter, and other scheduling-related information. These operations consume CPU time but do not directly perform the application's useful work.

Oversubscription can also cause **cache thrashing**. When different threads are frequently switched onto CPU cores, useful data and instructions in the CPU caches can be displaced by data belonging to other threads. The processor then has to retrieve data again from lower levels of the memory hierarchy, increasing memory-access latency.

As a result, increasing P beyond the available hardware parallelism can stop improving performance and may actually increase execution time because of context-switching overhead, thread state preservation, and cache effects.

Question 1.3 (Barriers)

In OpenMP, an implicit barrier exists at the end of a parallel region. The barrier ensures that all participating threads have completed the work inside the parallel region before execution proceeds beyond that region.

The barrier provides an important synchronization point between the worker threads and the subsequent sequential or parallel code.

Without this synchronization, some threads could continue executing into subsequent code blocks while other threads were still completing their previous work. This could create data hazards, including **race conditions**, inconsistent or incomplete results, and incorrect assumptions about whether shared data has already been updated.

Therefore, the implicit barrier ensures that the entire parallel team reaches a common synchronization point before the program continues to the next stage of execution.

Question 1.4 (Architectural Mapping)

A **hardware execution thread**, such as one provided by Hyper-Threading or SMT, is a hardware-level execution context supported by the processor. A physical CPU core can expose multiple logical processors to the operating system, allowing the processor to keep multiple execution contexts active.

An **operating system kernel thread** is a schedulable software entity managed by the operating system kernel. The OS scheduler assigns CPU time to kernel threads and can move them between available logical processors according to its scheduling decisions.

A **green thread or virtual thread** is a language/runtime-level abstraction. These threads are managed primarily by a language runtime or virtual machine rather than being directly equivalent to a physical CPU execution context. Multiple virtual threads can therefore be multiplexed over a smaller number of operating-system threads.

The three concepts exist at different abstraction levels:

Hardware thread → OS/kernel thread → language-level green/virtual thread

A hardware thread represents processor execution capacity, an OS thread represents a schedulable kernel entity, and a virtual/green thread represents a higher-level concurrency abstraction managed by a runtime.

task 2.1

=====

LAB 2 - NUMERICAL INTEGRATION / PI APPROXIMATION

=====

Number of integration steps: 100000000

CPU logical processors: 8

Running serial calculation...

Serial: Pi = 3.141592653590 | Time = 8.7010s | Error = 6.33e-13

Parallel benchmark:

Processes	Pi	Time (s)	Speedup	Error
Running with 1 processes...				
1	3.141592653590	8.8707	0.98x	6.33e-13
Running with 2 processes...				
2	3.141592653590	4.4991	1.93x	1.17e-13
Running with 4 processes...				
4	3.141592653590	2.8146	3.09x	1.11e-13
Running with 8 processes...				
8	3.141592653590	2.6310	3.31x	2.26e-14

=====
Benchmark completed.
=====

Results saved to:

lab2_pi_results.csv

Process finished with exit code 0